

How

Mathematically & Logically

Incapable Creatures See

CMP LC

Followed Dr. Manjusri Ishwara Lectures

(Actually Lecture Slides, we don't really have that type of hard subjects 😊)

Contents

Computation	3
Formal Languages	5
Syntax vs Semantics	6
String Operations	8
Kleene Closure (Σ^*)	9
Language (over an Alphabet Σ)	10
Language Definition Mechanisms	12
Why Do We Need Grammar?	15
Context-Free Grammar (CFG)	17
Regular Languages and Sri Lankan	25
Deterministic Finite Automata (DFA)	26
Regular Grammars	30
Language Operations	32
Nondeterminism	35
Rule of Regular Languages - Pumping Lemma	41
Regular Expressions	46
The Connection – Generalized NFA (GNFA)	50
Context-Free Languages (CFLs)	58
Ambiguity in Context-Free Grammars (CFGs)	61
Chomsky Normal Form (CNF)	63
Pushdown Automata	72
Equivalence of Context-Free Languages and Pushdown Automata	74
Pumping Lemma for CFLs	75
Turing Machines (TM)	78
Lambda Calculus	83

DISCLAIMER

This note was created based on lecture slides and my personal understanding . Therefore, its accuracy may not be 100%. Below are factual accuracy checks I made through multiple LLMs.

So, **use the notes at your own risk**. Good Luck!



Factual Accuracy Percentages

- Gemini 3 – 94%
- Claude Sonnet 4.6 – 90%
- GPT – 5.2 – 92%

All, the LLMs pointed there is an inconsistency in the symbol definition in Turing Machine compared to the standard way. So, please do your research on that.

Computational Models and Programming Language Concepts

This subject introduces the basic ideas behind how computers actually work at a theoretical level. Instead of focusing on programming, it explains how computation is done using symbols and fixed rules, without any human thinking or decision-making. We study formal languages to describe valid inputs, and different computational models like automata, grammars, and Turing machines to understand what computers can and cannot do. Overall, this subject helps build a clear foundation for understanding programming languages and the limits of computation.

Computation

What is Computation?

Computation is a **step-by-step process** that:

- Takes **inputs**
- Follows a **fixed set of rules**
- Produces **outputs**
- Uses **no judgment, creativity, or interpretation**

Every step must be **explicit** and **mechanically doable**.

Key Ideas

1. Step-by-step

- No skipping steps
- Same input → same output every time

2. Symbolic

- Inputs are **symbols** (numbers, characters, bits)
- Symbols have **no meaning by themselves**
- Meaning comes from **rules**, not understanding

3. Fixed Rules

- Rules never change
- Predefined set of steps

4. No Insight or Judgment

- No creativity
- No emotions
- No “feels right”

What CAN be Computation? (Examples)

- **Calculating the sum of first 100 natural numbers**
→ Clear steps, fixed formula
- **Cash register calculating a grocery bill**
→ Scan → add → total
- **Spam filter checking emails**
→ If rules match → spam

These all:

- Use symbols
- Follow rules
- Give predictable results

What is NOT Computation?

- **Mathematician choosing an interesting theorem**
→ Uses insight & creativity
- **Store owner giving a free item out of kindness**
→ Human judgment
- **Replying angrily to an email**
→ Depends on mood/emotion

If a human must “think” → NOT computation

If a machine can blindly follow rules → computation

Tiny Example

Input: $2 + 3$

Rules:

1. Take two numbers
2. Apply addition
3. Output result

Output: 5

No thinking. Just rules.

Formal Languages

What is a Formal Language?

A **formal language** is a set of rules used to describe **valid strings of symbols**.

Programming languages are formal languages designed to **describe** solutions to problems and also **implement** them on a computer.

A programming language must:

- Be **readable** (easy to understand)
- Be **writable** (easy to use)
- Have clear **rules**

Since a language cannot describe itself, a **meta-language** is used to describe it.

Example: Grammar rules used to define a programming language (used by compilers).

Alphabet (Σ)

An **alphabet (Σ)** is:

- A **finite**
- **Non-empty** set of symbols

Symbols have **no meaning by themselves**.

Example:

- $\Sigma = \{a, b, c\}$
- a, b, c are symbols
- Σ cannot be empty

Word / String

- A **word (string)** is a **finite sequence of symbols** from Σ
- Two strings are equal if symbols and order are the same

Example:

- $\Sigma = \{a, b\}$
- $w = aabb$
- $|w| = 4$ (length)
- ϵ = empty string (length 0)

Syntax vs Semantics

Syntax (Form / Structure)

Syntax defines **how symbols can be combined**.

- Concerned only with **structure**, not meaning
- Decides whether something is **well-formed**

Two parts of syntax:

1. Lexical Syntax

- Smallest meaningful units (tokens)
- Examples: keywords, identifiers, numbers

2. Phrase-Structure Syntax

- How tokens combine into valid statements
- Examples: if blocks, loops, brackets

Syntax analysis checks:

- Are brackets correct?
- Is the format valid?

Semantics (Meaning / Behavior)

Semantics defines the **meaning** of syntactically correct sentences.

- In programming languages, semantics describes:
 - What the computer **does** when executing the program

Important:

A program can be syntactically correct but semantically meaningless.

Example:

- “My mother’s white cat is black.”
 - Syntax ✓
 - Semantics ✗

Syntax vs Semantics Order

- **Syntax analysis comes first**
- **Semantic analysis comes after**

Syntax is easier and cheaper to check than meaning.

Notation

- Lowercase letters **a, b, c, ...** → symbols in Σ
- Lowercase letters **u, v, w, ...** → strings (words)

(Just a convention for convenience as I believe)

Just again...

- **Alphabet** → symbols
- **Word** → symbols in order
- **Syntax** → is it written correctly?
- **Semantics** → does it make sense?

String Operations

Concatenation

- **Definition:** Joining two strings end-to-end.
- **Notation:** If $w = ab$ and $v = cd$, then $wv = abcd$

Reverse

- **Definition:** Writing the symbols in **reverse order**.
- **Example:** $w = abc \rightarrow \text{reverse}(w) = cba$

Length

- **Definition:** Number of symbols in the string
- **Notation:** $|w|$
- **Example:** $w = abc \rightarrow |w| = 3$

Empty String

- **Symbol:** ϵ
- **Definition:** String of length 0
- **Properties:**
 - $|\epsilon| = 0$
 - $\epsilon w = w \epsilon = w$ (joining empty string does nothing)

Prefix & Suffix

- If $w = uv$
 - $u = \text{prefix}$
 - $v = \text{suffix}$
- **Example:** $w = abcd \rightarrow u = ab$ (prefix), $v = cd$ (suffix)

Repetition

- If w is a string, $w^n = w$ repeated n times
- **Example:** $w = ab, w^3 = ababab$

Strings of Fixed Length

- Σ^k = all strings of length k formed using symbols from Σ
- **Example:** $\Sigma = \{0, 1\}, k = 2 \rightarrow \Sigma^2 = \{00, 01, 10, 11\}$

Kleene Closure (Σ^*)

Kleene Star (Σ^*)

- **Definition:** The set of all strings you can make by concatenating **zero or more symbols** from Σ .
- **Formally:**
$$\Sigma^* = \{\epsilon\} \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$
 - ϵ = empty string
 - Σ^1 = strings of length 1
 - Σ^2 = strings of length 2
 - ... and so on

Example:

- $\Sigma = \{a, b\}$
- $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$

Infinite set because you can keep concatenating symbols forever

Positive Closure (Σ^+)

- **Definition:** Like Σ^* , but **no empty string**
- **Formally:** $\Sigma^+ = \Sigma^* - \{\epsilon\}$

Example:

- $\Sigma = \{a, b\}$
- $\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, \dots\}$

$\Sigma^* \rightarrow$ zero or more (includes empty string)

$\Sigma^+ \rightarrow$ one or more (at least one symbol)

Language (over an Alphabet Σ)

What is a Language?

- A **language L** over an alphabet Σ is just a **subset of Σ^*** .

Basically: any collection of strings you can make from the alphabet.

- **It can be:**
 - Empty $\rightarrow L = \{\}$
 - Finite $\rightarrow L = \{a, b, ab\}$
 - Infinite $\rightarrow L = \{\epsilon, a, b, aa, ab, ba, \dots\}$
- **Note:**
 - Different sets can give the **same language** after applying Kleene star.
 - $S^* = (S^*)^* \rightarrow$ applying star twice doesn't change anything.

Example:

- $\Sigma = \{a, b\}$
- $L = \{\epsilon, a, b, aa, ab, ba, bb, \dots\} \rightarrow L = \Sigma^*$

Operations on Languages

Since languages are sets of strings, you can use **set operations**:

1. **Union ($L1 \cup L2$)** $\rightarrow$ all strings in $L1$ or $L2$
 - Example:
 - $L1 = \{a, ab\}, L2 = \{b, aa\}$
 - $L1 \cup L2 = \{a, ab, b, aa\}$
2. **Intersection ($L1 \cap L2$)** $\rightarrow$ strings common to both
 - Example:
 - $L1 = \{a, ab\}, L2 = \{ab, b\}$
 - $L1 \cap L2 = \{ab\}$
3. **Difference ($L1 - L2$)** $\rightarrow$ strings in $L1$ but not in $L2$
 - Example:
 - $L1 = \{a, ab\}, L2 = \{ab, b\}$
 - $L1 - L2 = \{a\}$
4. **Complement (L')** $\rightarrow$ strings in Σ^* **not in L**
 - Example:

- $\Sigma = \{a, b\}$, $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
- $L = \{a, b\}$
- $L' = \{\epsilon, aa, ab, ba, bb, \dots\}$

5. **Concatenation ($L1 L2$)** $\rightarrow$ take every string from $L1$ and join with every string from $L2$

○ Example:

- $L1 = \{a, b\}$, $L2 = \{x, y\}$
- $L1 L2 = \{ax, ay, bx, by\}$

Language Definition Mechanisms

What does it mean to “define” a language?

To define a language, we must specify **which strings are acceptable** and which are not. This is done by giving a **set of rules** over an alphabet Σ .

A language **L** over Σ is a subset of Σ^* , but for **complex languages**, simple set notation is not enough. So we use **rule-based mechanisms**.

Two Ways to Define a Language

1. Recognizers (Membership Checking)

These rules are used to **check** whether a given string belongs to the language.

- Input: a string
- Output: **YES / NO**

“Is this string in the language?”

Examples:

- DFA / NFA
- Turing Machine
- Program that validates an email

Example language:

L = all strings over $\{0,1\}$ that end with 01

- Input: 1101 $\rightarrow$ ACCEPT
- Input: 1110 $\rightarrow$ REJECT

2. Generators (String Creation)

These rules are used to **generate all valid strings** of the language.

- Start from rules
- Produce valid strings

“How can I build strings in the language?”

Examples:

- Grammars (Regular Grammars, CFGs)

Example grammar:

$S \rightarrow aS \mid b$ (If you cannot understand this statement, please look at Explanation 01 below.)

This generates:

b, ab, aab, aaab, ...

Explanation 01

How to Read This Rule

$S \rightarrow aS \mid b$

Read it as:

“S can be replaced by aS OR b.”

That’s it. Nothing fancy.

What the Symbols Mean

- **S** $\rightarrow$ a *non-terminal symbol* (a variable / placeholder)
- $\rightarrow$ $\rightarrow$ “can be replaced by”
- **aS** $\rightarrow$ symbol a followed by S
- **|** $\rightarrow$ “or”
- **b** $\rightarrow$ symbol b

How It Generates Strings

Start with **S** and keep replacing using the rule.

Example derivations:

1. **$S \rightarrow b$**
- string = b
2. **$S \rightarrow aS \rightarrow ab$**
- string = ab
3. **$S \rightarrow aS \rightarrow aS \rightarrow aab$**
- string = aab
4. **$S \rightarrow aS \rightarrow aS \rightarrow aS \rightarrow aaab$**
- string = aaab

What Language Does This Describe?

All strings that:

- Have **zero or more a**
- End with **one b**

So the language is:

{ b, ab, aab, aaab, ... }

Recognizer vs Generator

Recognizer	Generator
Checks strings	Creates strings
YES / NO answer	Produces valid words
Automata	Grammars

Why Not Just Use Set Notation?

- Set notation works for **simple languages**
- Complex languages are:
 - Infinite
 - Structured
- Writing them as sets is **impractical**

That's why we use **machines and grammars**.

Recognizer → “Is this string valid?”

Generator → “How do I make valid strings?”

Before moving on, let's see what the grammar is...

Why Do We Need Grammar?

How does a compiler know if a program is correct?

A compiler does **not understand meaning** like humans do. It simply **checks rules**.

Example:

- `int x = 10;` → follows rules
- `= int 10 x;` → breaks rules

So correctness is about **structure**, not intention.

Are Programming Languages Infinite?

Yes. A programming language has **infinitely many valid programs**.

Why infinite?

- Unlimited identifiers (`x`, `totalSum`, `value123`)
- Unlimited numbers (`1`, `10`, `1000`, `99999`)
- Unlimited combinations of statements

But the language itself is defined using a **finite description**.

Big Question: How can we describe infinitely many programs using a finite description?

Answer: **Using rules**

These rules specify:

- What is allowed
- What is not allowed
- How symbols can be combined

These rules together are called a Grammar

What is Grammar?

A grammar is:

- A **finite set of rules**
- That describes **valid strings** of a language

What do rules tell us?

- What symbols are allowed
- The order they can appear
- How small parts form bigger structures

What is a String in Programming?

A string generated by grammar can be:

- A number
- An expression
- A statement
- A full program

Infinite Programs from Finite Rules

Even with a few rules, grammar can generate **infinite strings**.

Example: Grammar for Numbers

<digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<number> → <digit> | <digit><number>

Example: Generating 34

Start:

<number>

Steps:

<number>

⇒ <digit><number>

⇒ 3<number>

⇒ 3<digit>

⇒ 34

Example: Generating 246

<number>

⇒ <digit><number>

⇒ 2<number>

⇒ 2<digit><number>

⇒ 24<number>

⇒ 24<digit>

⇒ 246

✓ Same rules

✓ Different results

✓ Infinite possibilities

Context-Free Grammar (CFG)

What is a CFG?

A **Context-Free Grammar (CFG)** is the **formal way** we describe the **syntax** (structure) of programming languages.

In simple words:

CFG tells us **how valid programs are written**, not what they *mean*.

Formal Definition of CFG

A CFG is defined as:

$$G = (V, \Sigma, P, S)$$

Don't panic, this is just a list of parts.

1. V — Non-terminal Symbols

Non-terminals:

- Are **not actual symbols** of the language
- Act like **placeholders** or **variables**
- Used to describe structure

Examples:

- $\langle \text{expr} \rangle \rightarrow$ an expression
- $\langle \text{stmt} \rangle \rightarrow$ a statement
- $\langle \text{term} \rangle \rightarrow$ part of an expression

These help us **build rules**, but they never appear in the final program.

2. Σ — Terminal Symbols

Terminal symbols:

- Are the **actual symbols** of the language
- Appear in the **final program**
- Cannot be replaced further

In programming languages, terminals include:

- Keywords (if, while, int)
- Operators (+, *, =)
- Identifiers (x, total)
- Numbers (10, 42)

Examples:

- +
- =
- *
- A, B, C

Easy Way to Remember

- **Non-terminals (V)** → ideas / structure
- **Terminals (Σ)** → real code symbols

Example

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \text{number}$

- $\langle \text{expr} \rangle$ and $\langle \text{term} \rangle \rightarrow$ non-terminals
- + and number $\rightarrow$ terminals

Final result (valid string): number + number

Production Rules (P)

Production rules tell us **how non-terminals can be replaced**.

General form:

Non-terminal $\rightarrow$ symbols

- Left side $\rightarrow$ **one non-terminal**
- Right side $\rightarrow$ terminals and/or non-terminals

Example:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$

This means:

An expression can be an expression plus a term.

Start Symbol (S)

- The **start symbol** is the symbol where **derivation always begins**
- It represents the **entire program or structure**
- Usually denoted by **S**

Think of it as the **entry point** of the grammar.

What is BNF?

BNF (Backus–Naur Form) is just a **standard way of writing grammar rules**.

It makes grammar rules:

- Clear
- Unambiguous
- Easy for compilers to read

BNF Example (Assignment Statement)

Grammar Rules

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

- $\langle \text{assign} \rangle, \langle \text{expr} \rangle, \langle \text{id} \rangle \rightarrow$ non-terminals
- $=, +, A, B, C \rightarrow$ terminals

Example Derivation

Goal: Generate the string

$A = B + C$

Step-by-step:

$\langle \text{assign} \rangle$

$\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{id} \rangle + \langle \text{id} \rangle$

$\Rightarrow A = B + C$

✓ All non-terminals replaced

✓ Only terminals remain

✓ Valid string

In case it is not clean, look at below example:

CFG Example: Assignment with Expressions

Grammar Rules

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

- **Non-terminals:** $\langle \text{assign} \rangle, \langle \text{expr} \rangle, \langle \text{term} \rangle, \langle \text{factor} \rangle, \langle \text{id} \rangle$

- **Terminals:** =, +, *, A, B, C, (,)
- **Start symbol:** <assign>

Step-by-Step Derivation of $A = B + C * A$

We start with <assign>:

1. <assign>
 $\Rightarrow \langle id \rangle = \langle expr \rangle$
(Start symbol replaced by assignment pattern)
2. $\langle id \rangle = \langle expr \rangle$
 $\Rightarrow A = \langle expr \rangle$
(Choose $\langle id \rangle \rightarrow A$)
3. $A = \langle expr \rangle$
 $\Rightarrow A = \langle expr \rangle + \langle term \rangle$
(Use $\langle expr \rangle \rightarrow \langle expr \rangle + \langle term \rangle$)
4. $A = \langle expr \rangle + \langle term \rangle$
 $\Rightarrow A = \langle term \rangle + \langle term \rangle$
(Replace $\langle expr \rangle$ with $\langle term \rangle$ since $\langle expr \rangle \rightarrow \langle term \rangle$ is allowed)
5. $A = \langle term \rangle + \langle term \rangle$
 $\Rightarrow A = \langle factor \rangle + \langle term \rangle$
(Replace $\langle term \rangle \rightarrow \langle factor \rangle$)
6. $A = \langle factor \rangle + \langle term \rangle$
 $\Rightarrow A = B + \langle term \rangle$
(Choose $\langle factor \rangle \rightarrow \langle id \rangle$ and $\langle id \rangle \rightarrow B$)
7. $A = B + \langle term \rangle$
 $\Rightarrow A = B + \langle term \rangle * \langle factor \rangle$
*(Use $\langle term \rangle \rightarrow \langle term \rangle * \langle factor \rangle$)*
8. $A = B + \langle term \rangle * \langle factor \rangle$
 $\Rightarrow A = B + \langle factor \rangle * \langle id \rangle$
(Replace $\langle term \rangle \rightarrow \langle factor \rangle$ and $\langle factor \rangle \rightarrow \langle id \rangle$)
9. $A = B + \langle factor \rangle * \langle id \rangle$
 $\Rightarrow A = B + C * A$
(Choose $\langle factor \rangle \rightarrow \langle id \rangle$, $\langle id \rangle \rightarrow C$, and $\langle id \rangle \rightarrow A$ for the last part)

All non-terminals replaced $\rightarrow$ valid string

Key Points

- **Leftmost derivation:** Always replace the **leftmost non-terminal** first (like in the example above).
- **Rightmost derivation:** You could also start replacing from the **rightmost non-terminal**.

- **Parse tree:** You can draw a tree showing how `<assign>` splits into `<id>` and `<expr>`, `<expr>` splits into `<term>` and so on — helps visualize the structure.

Parse Trees (CFG)

What is a Parse Tree?

- A **hierarchical representation** of how a string is generated from a grammar
- Shows **how grammar rules are applied step by step**

Structure

- **Root** → Start symbol (S or `<assign>`)
- **Internal nodes** → Non-terminals (`<expr>`, `<term>`, `<factor>`)
- **Leaves** → Terminals (A, =, +, (, B, C,))

Example: String A = (B + C) + A

CFG Rules Used:

`<assign>` → `<id>` = `<expr>`

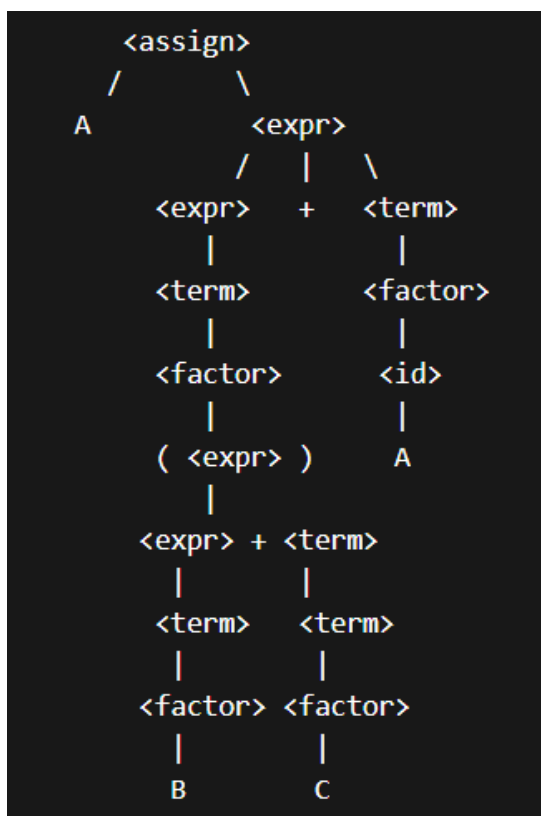
`<expr>` → `<expr>` + `<term>` | `<term>`

`<term>` → `<term>` * `<factor>` | `<factor>`

`<factor>` → (`<expr>`) | `<id>`

`<id>` → A | B | C

Parse Tree (Text Version):



Key Points

- **Start at the root**, follow **production rules**, replace **non-terminals** until only terminals remain
- **Leaves = final string**
- Parse trees are used in **compilers** to check syntax

Chomsky Hierarchy of Grammars

Grammars have **different levels of power**, just like computational machines.

This classification is called the **Chomsky Hierarchy**.

Type	Name	Recognized by	What it can describe
3	Regular Grammar	Finite Automaton	Simple patterns like keywords, identifiers, numbers
2	Context-Free Grammar	Pushdown Automaton	Nested structures, balanced expressions, arithmetic expressions, if-else blocks, function definitions
1	Context-Sensitive Grammar	Linear Bounded Automaton (LBA)	Rules where context matters
0	Unrestricted Grammar	Turing Machine	Most general; any computable language

Key Points

- **Type 3 (Regular Grammar)**
 - Simple patterns, no nesting
 - Examples: keywords, variable names, numbers
- **Type 2 (Context-Free Grammar)**
 - Handles **nested structures and recursion**
 - Examples: arithmetic expressions, if-else blocks, function definitions
 - **Most programming languages are Type 2** because of nested parentheses, nested blocks, and expressions inside expressions
- **Type 1 (Context-Sensitive Grammar)**
 - More powerful than CFG
 - Can describe languages where **context matters**
 - Rarely needed in programming languages
- **Type 0 (Unrestricted Grammar)**
 - Equivalent in power to a **Turing Machine**

- Can describe any computable language

Ambiguity in Grammars

Ambiguity happens when **the same string can have more than one parse tree**:

- More than one parse tree → more than one structure → more than one meaning
- **Dangerous for compilers** → may produce incorrect results

Example (English):

'I saw the man with the telescope.'

- Who has the telescope?
- Two meanings → sentence is ambiguous

How to Solve Ambiguity

- **Control the structure** of the grammar
- Separate operators using different **non-terminals**
- Enforce **operator precedence** in the grammar

Example: Arithmetic Operators

Original ambiguous grammar:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow \text{id}$

- Ambiguity: $A + B * C$ can be parsed as either:

1. $(A + B) * C$

2. $A + (B * C)$

Solution:

- Keep ***** **higher precedence** than **+** by separating $\langle \text{term} \rangle$ and $\langle \text{expr} \rangle$
- $\langle \text{expr} \rangle$ handles **+**
- $\langle \text{term} \rangle$ handles *****
- $\langle \text{factor} \rangle$ handles identifiers (id)

Now the parse tree is **unique** → no ambiguity

Operator Associativity

Associativity decides **the order of evaluation** when operators have the same precedence.

- **Left-associative** → evaluated **left to right**
 - Example: $A - B - C = (A - B) - C$

- **Right-associative** → evaluated **right to left**

- Example: $A = B = C = A \rightarrow (B = C) = A$

Precedence vs Associativity

Concept	What it decides
Precedence	Which operator is done first (* vs +)
Associativity	Order of evaluation when operators have same precedence

Both are enforced by grammar structure in a compiler.

Common Operators

Operator	Associativity	Example
Exponential ^	Right to left	$a \wedge b \wedge c$
Multiplication *, Division /	Left to right	$a * b / c$
Addition +, Subtraction -	Left to right	$A - B + C$

Remember:

“Same precedence → follow associativity. Left → left to right, Right → right to left.”

Regular Languages and Sri Lankan

Regular languages are about checking format, not meaning.

Examples from Sri Lanka where format is checked:

- **NIC number** (old/new format)
→ checks length, digits, letters like V or X
- **Mobile numbers**
→ must start with 07 and have 10 digits
- **Vehicle number plates**
→ pattern like ABC-1234
- **Postal codes**
→ exactly 5 digits

In all these:

- The system **does NOT care about meaning**
- It only checks **pattern / structure**

Does the checker need to remember earlier input?

- **No**
- It only needs to know:
 - “What did I see just now?”
 - “Which state am I in?”

Example:

- For a mobile number → just count digits
- For NIC → check character order

Can it handle very long inputs without writing anything down?

- **Yes**
- No memory
- No lists
- No notes
- Just **current state**

That's exactly the power of **Finite Automata**

Deterministic Finite Automata (DFA)

A **DFA** is a machine that:

- Reads input **one symbol at a time**
- Moves between **states**
- Accepts or rejects the string at the end

Formal Definition of DFA

A DFA is written as a **5-tuple**:

$(Q, \Sigma, \delta, q_0, F)$

Symbol	Meaning
Q	Finite set of states
Σ	Alphabet (input symbols)
δ	Transition function
q_0	Start state
F	Set of accept (final) states

Important DFA Rules

- Exactly **one transition** for every **(state, symbol)** pair
- No guessing, no branching
- Deterministic = predictable
- Input is accepted **only if** DFA ends in an **accept state**

Example 1: DFA that accepts strings containing “01”

Alphabet: $\{0,1\}$

Idea:

- Keep moving until you see 0
- After 0, if you see 1 → ACCEPT
- Once accepted, stay accepted

States idea:

- q_0 → start (haven't seen 0)
- q_1 → seen 0
- q_2 → seen 01 ☒ (accept)

This DFA:

- Works for **any length**
- Needs **no memory**
- Only tracks **current state**

Example 2: Sum is 0 or multiple of 3

Alphabet: {0,1,2,3}

Key trick:

- DFA tracks **remainder mod 3**
- States:
 - $q_0 \rightarrow \text{sum} \equiv 0 \pmod{3}$ ☒
 - $q_1 \rightarrow \text{sum} \equiv 1$
 - $q_2 \rightarrow \text{sum} \equiv 2$

Every new digit:

- Update remainder
- Move state accordingly

This shows:

- DFA can do **simple arithmetic**
- But only with **limited memory**

Big Picture

- Regular languages check **pattern, not meaning**
- DFA:
 - No memory
 - No stack
 - No writing things down
- Perfect for:
 - NICs
 - Phone numbers
 - IDs
 - Keywords
 - Simple validations

Formal Definition of Computation (DFA)

We are given:

- $FA = (Q, \Sigma, \delta, q_0, F) \rightarrow$ a finite automaton
- $w \rightarrow$ an input string made only of symbols from Σ

When does a DFA accept a string w ?

A DFA **accepts** a string w **if it can process the string symbol by symbol and end in an accept state.**

Formally, this happens if there exists a sequence of states:

$s_0, s_1, s_2, \dots, s_n$

that satisfies **three conditions** 

1. Start correctly

$$s_0 = q_0$$

- The computation **must start at the start state**

2. Follow the transition rules

$$\delta(s_i, w_{i+1}) = s_{i+1} \quad \text{for } i = 0 \text{ to } n-1$$

Meaning:

- Read the input **one symbol at a time**
- Use the transition function δ
- Move from the current state to the next state

No guessing. No skipping. Just follow the rules.

3. End in an accept state

$$s_n \in F$$

- After reading the **entire string**
- The final state **must be an accept state**

If yes $\rightarrow$ string is accepted

If not $\rightarrow$ string is rejected

What does “FA recognizes a language” mean?

The language recognized by FA is:

$$L = \{ w \in \Sigma^* \mid \text{FA accepts } w \}$$

In plain English:

The language L is the set of all strings that the automaton accepts.

Tiny Example

Alphabet:

$$\Sigma = \{0, 1\}$$

Language:

L = all strings that contain "01"

Then:

- 00110 $\rightarrow$ accepted
- 1111 $\rightarrow$ rejected

So:

$$L = \{ w \in \Sigma^* \mid \text{DFA accepts } w \}$$

Summary

- DFA computation = **walk through states**
- Read input **left to right**
- Follow **δ rules**
- End in **accept state**
- All accepted strings together = **the language**

Regular Grammars

A **regular grammar** is a grammar with **very strict rule shapes**.
Because of these restrictions, it has the **same power as a DFA**.

Formally:

A regular grammar is

$G = (N, \Sigma, P, S)$

Where:

- **N** $\rightarrow$ non-terminals
- **Σ** $\rightarrow$ terminals
- **P** $\rightarrow$ production rules
- **S** $\rightarrow$ start symbol

Forms of Production Rules

A regular grammar can be **ONLY ONE** of the following types (not both):

Right-Linear Grammar

Rules look like:

$A \rightarrow aB \mid a \mid \varepsilon$

Meaning:

- Terminal comes **first**
- Non-terminal (if any) comes **to the right**

Example:

$S \rightarrow aA$

$A \rightarrow bS \mid b$

Left-Linear Grammar

Rules look like:

$A \rightarrow Ba \mid a \mid \varepsilon$

Meaning:

- Non-terminal comes **first**
- Terminal comes **to the right**

Example:

$S \rightarrow Aa$

$A \rightarrow Sb \mid b$

Important Rule

- You **cannot mix** right-linear and left-linear rules in the same grammar.

Language Generated by a Regular Grammar

The language generated by grammar **G** is:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

Plain English:

All strings that can be derived from the start symbol using the grammar rules.

Regular Grammar → Regular Language

- If **G** is a regular grammar
- Then **L(G)** is a regular language

What is a Regular Language?

A language is **regular** if:

Some finite automaton (DFA/NFA) can recognize it.

Another simple way to say it:

A regular language can be checked **without remembering the whole past**.

Intuition

Regular languages:

- Don't need memory
- Don't need stacks
- Don't need recursion

They only depend on:

- **Current state**
- **Current input symbol**

LK Real-Life Feel

Checking:

- NIC format
- Phone number pattern
- Email pattern

Pattern → yes

Meaning → irrelevant

Language Operations

Think of a **language** as a **set of strings**.

So all language operations are just **set operations**, but applied to strings.

Let:

$$\Sigma = \{a, b\}$$

$$A = \{a, ab\}$$

$$B = \{b, ab\}$$

Union ($A \cup B$)

Definition:

$$A \cup B = \{x \in \Sigma^* \mid x \in A \text{ or } x \in B\}$$

Meaning:

All strings that are in **A or B (or both)**.

Example:

$$A = \{a, ab\}$$

$$B = \{b, ab\}$$

$$A \cup B = \{a, b, ab\}$$

Intersection ($A \cap B$)

Definition:

$$A \cap B = \{x \in \Sigma^* \mid x \in A \text{ and } x \in B\}$$

Meaning:

Only strings that are **common to both**.

Example:

$$A \cap B = \{ab\}$$

Concatenation ($A \circ B$)

Definition:

$$A \circ B = \{xy \in \Sigma^* \mid x \in A \text{ and } y \in B\}$$

Meaning:

Take **one string from A**, then **one from B**, and stick them together.

Example:

$$A = \{a, ab\}$$

$$B = \{b\}$$

$$A \circ B = \{ab, abb\}$$

- $a + b = ab$
- $ab + b = abb$

Kleene Closure (A^*)

Definition:

$$A^* = \{ x_1 x_2 x_3 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A \}$$

Meaning:

- Concatenate strings from A **zero or more times**
- Includes ϵ (**empty string**)

Example:

$$A = \{a\}$$

$$A^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$

Usually **infinite**.

Complement (A^c)

Definition:

$$A^c = \{ x \in \Sigma^* \mid x \text{ not in } A \}$$

Meaning:

All strings over Σ that are **NOT in A**.

Example:

$$\Sigma = \{a, b\}$$

$$A = \{a\}$$

$$A^c = \{\epsilon, b, aa, ab, ba, bb, \dots\}$$

Important:

- Complement is **always defined with respect to Σ^***

Closure Property

Definition:

A class of languages is **closed under an operation** if applying that operation to languages in the class gives a language **still in the same class**.

Closure of Regular Languages

Regular languages are closed under:

- Union

- Intersection
- Complement
- Concatenation
- Kleene Star

Why?

Because:

- We can **build new DFAs** from old DFAs
- Using product construction, extra states, etc.

Result is **still a DFA**

So result is **still a regular language**

Nondeterminism

Deterministic (DFA)

“I always know exactly where I will go next.”

- One state
- One input symbol
- **Exactly one choice**

Like Google Maps with **only one road**.

Nondeterministic (NFA)

“I may have multiple choices... I’ll try all of them.”

- One state
- One input symbol
- **Zero, one, or many choices**

Like standing at a junction and **cloning yourself** to try all roads.

Important idea

Nondeterminism is a generalization of determinism

That means:

- Every DFA is automatically an NFA
- But not every NFA is a DFA

Why?

- DFA = special case of NFA with **exactly one choice**

DFA vs NFA

Feature	DFA	NFA
Choices per symbol	Exactly one	Zero, one, or many
ϵ (empty) moves	No	☒ Yes
Guessing	No	☒ Yes
Power	Same	Same
Easier to design	No	Yes

ϵ -transitions

NFA can:

Move from one state to another **without reading any input**

This is called an **ϵ -transition**.

Meaning:

- “I’ll change state for free”
- Input pointer doesn’t move

How does an NFA compute?

This is the MOST important conceptual question.

Think like this:

- NFA reads input
- Whenever it has choices $\rightarrow$ it **branches**
- It explores **all possible paths at the same time**
- If **at least one path** ends in an accept state $\rightarrow$ ACCEPT

All paths reject $\rightarrow$ REJECT

At least one path accepts $\rightarrow$ ACCEPT

Formal Definition of NFA

An NFA is still a **5-tuple**:

$(Q, \Sigma, \delta, q_0, F)$

Only **one thing changes**

Transition Function

$\delta : Q \times \Sigma_\epsilon \rightarrow P(Q)$

Meaning:

- Given a state and a symbol (or ϵ)
- NFA returns a **set of states**, not just one

$P(Q)$ = power set = “set of all possible states”

Example 1

“0 in the 4th position from the end”

NFA strategy:

- The NFA *guesses* which position is the 4th from the end

- From that point, it verifies:
 - the symbol is 0
 - exactly three symbols remain afterward

Why DFA is hard:

- While reading input, a DFA does not know where the string will end
- To identify the 4th-from-last symbol, it would need to know how many symbols remain
- This effectively requires remembering unbounded information, which a DFA cannot do easily

Why NFA works:

- The NFA can try all possible guesses in parallel
- If **any one path** makes the correct guess, the string is accepted

Example 2

Even number of bs and no single unmatched b

Idea:

- Track whether the count of bs is even or odd
- Ensure the computation ends in a state where:
 - the number of bs is even
 - the string does not end with an unmatched b

Why this is manageable:

- The condition depends only on the current situation
- A small number of states is enough
- No need to remember the full input history
- State transitions alone are sufficient

Similarity

NFA and DFA recognize exactly the same class of languages — regular languages

So why NFA?

- Easier to **design**
- Easier to **think**
- Then later → convert NFA → DFA

How to Convert an NFA to a DFA

The Idea

- **NFA:** Can be in multiple states at the same time because it can “guess.”
- **DFA:** Can only be in **one state at a time**.
- So the trick: **make each DFA state represent a set of NFA states**.

Think of it like a **team of NFA states moving together as one DFA state**.

Step 1: Start with ϵ -Closure

- Some NFA states can move **without reading anything** (ϵ transitions).
- Compute the **ϵ -closure** of the start state.
- That **entire set becomes the DFA's start state**.

Simple way to think: “If I start here, where can I be immediately without input?”

Step 2: Process Each Input Symbol

- For each DFA state (a set of NFA states):
 1. Pick an input symbol (0, 1, etc.)
 2. See where **all NFA states in this set would go** on that input
 3. Take **ϵ -closure of the results** — that's your **next DFA state**

Intuition: “After reading this symbol, where could the NFA be?”

- Remember: a DFA **cannot guess**, so it treats the set as a single state.

Step 3: Mark Accepting States

- Any DFA state that **includes at least one NFA accepting state** becomes an **accepting DFA state**.

Intuition: “If any NFA path leads to accept, the DFA accepts too.”

Step 4: Repeat

- Keep doing Step 2 for **all new DFA states** until no new states appear.

Intuition: “We've tracked all possible NFA moves, so the DFA is complete.”

Step 5: Done

- You now have a DFA that behaves exactly like the NFA but **never guesses**.
- Each DFA state is just a **set of possible NFA states at that point**.

NFA to DFA Example

States: q_0, q_1, q_2

Start state: q_0

Accepting state: q_2

Alphabet: $\{a, b\}$

Transitions:

- $q_0 \rightarrow \epsilon \rightarrow q_1$
- $q_1 \rightarrow a \rightarrow q_1$
- $q_1 \rightarrow b \rightarrow q_2$

Step 1: Compute ϵ -Closure of Start State

- Start at **q_0** $\rightarrow$ include q_0 itself $\rightarrow \{q_0\}$
- $q_0 \rightarrow \epsilon \rightarrow q_1 \rightarrow$ add $q_1 \rightarrow \{q_0, q_1\}$
- $q_1 \rightarrow$ no $\epsilon \rightarrow$ stop

DFA start state = $\{q_0, q_1\}$

Intuition: Before reading anything, the NFA could be in either q_0 or q_1 .

Step 2: Process Each Input Symbol

From DFA state $\{q_0, q_1\}$:

1. Input a:

- $q_0 \rightarrow$ no $a \rightarrow$ ignore
- $q_1 \rightarrow a \rightarrow q_1 \rightarrow \epsilon\text{-closure}(q_1) = \{q_1\}$

So $\{q_0, q_1\} \xrightarrow{a} \{q_1\}$

2. Input b:

- $q_0 \rightarrow$ no $b \rightarrow$ ignore
- $q_1 \rightarrow b \rightarrow q_2 \rightarrow \epsilon\text{-closure}(q_2) = \{q_2\}$

So $\{q_0, q_1\} \xrightarrow{b} \{q_2\}$

Next, from DFA state $\{q_1\}$:

- Input $a \rightarrow q_1 \rightarrow a \rightarrow q_1 \rightarrow \epsilon\text{-closure}(q_1) = \{q_1\}$
- Input $b \rightarrow q_1 \rightarrow b \rightarrow q_2 \rightarrow \epsilon\text{-closure}(q_2) = \{q_2\}$

From DFA state $\{q_2\}$:

- $q_2 \rightarrow$ no transitions $\rightarrow$ stay $\{\}$ (dead state if needed)

Step 3: Mark Accepting States

- DFA states that **include NFA accepting state q_2** $\rightarrow \{q_2\}$

$\{q_2\}$ is an accepting DFA state

Step 4: Repeat Until Complete

- All DFA states now: $\{q_0, q_1\}, \{q_1\}, \{q_2\} \rightarrow$ done, no new sets appear

Step 5: Done – DFA Table

DFA State	a	b	Accept?
$\{q_0, q_1\}$	$\{q_1\}$	$\{q_2\}$	No
$\{q_1\}$	$\{q_1\}$	$\{q_2\}$	No
$\{q_2\}$	$\{\}$	$\{\}$	Yes

Each DFA state = a **set of NFA states**.

No guessing, completely deterministic.

Rule of Regular Languages- Pumping Lemma

Some languages **look regular**—like they might be handled by a DFA—but they actually aren't.

- Example: $0^n 1^n \rightarrow$ any number of 0's followed by exactly the same number of 1's.
- Problem: a finite automaton **can't remember** how many 0's it saw.

So we need a **test to catch non-regular languages**.

What is the Pumping Lemma?

- It's a **rule every regular language must follow**.
- If a language **fails this rule**, it **cannot be regular**.
- Basically: for **long enough strings**, parts of the string can be **repeated ("pumped")** without breaking the language.

Think: "If a language is regular, I can take a middle part of a long string and repeat it, and it still works."

What the Pumping Lemma is REALLY saying

If a language is regular, then **long enough strings** in that language have a **repeatable middle part**.

That's it. Everything else is just formal wording.

Meaning of the symbols (These are the symbols used in Pumping Lemma)

1. Pumping length P

- P is **some number**
- You **don't know its value**
- It comes from the DFA
- You just assume it exists

2. String length |S|

- |S| = number of symbols in S

Example:

$$\begin{array}{l} S = aabb \\ |S| = 4 \end{array}$$

3. Splitting $S = xyz$

This means:

$$x + y + z = S$$

Example:

```
S = abcde
x = ab
y = c
z = de
```

Now the THREE CONDITIONS

1. Condition 1: $xy^iz \in A$ for every $i \geq 0$

This means:

- You can **repeat y**
- Or **remove y**
- And the string must **still be in the language**

Examples:

```
i = 0 → xz
i = 1 → xyz
i = 2 → xyyz
```

2. Condition 2: $|y| > 0$

- **y cannot be empty**
- It must contain **at least one symbol**
Why?
If y were empty, repeating it does nothing → useless

3. Condition 3: $|xy| \leq P$

This is CRITICAL.

It means:

- x and y together are **inside the first P symbols**
- y is **near the beginning** of S
- y **cannot reach far into the string**

This limits **where y can be**

Strategy to PROVE a language is NOT regular

You are trying to **break** these rules.

Steps:

1. **Assume** the language is regular (even though it's not)
2. Let **P** be the pumping length
3. Choose a **carefully designed string S**
4. Force **y** to be in a bad place
5. Show pumping **breaks the language**
6. Contradiction $\rightarrow$ language is NOT regular

Now the actual example

Language:

$$A = \{ a^n b^n \mid n \geq 0 \}$$

Meaning:

- All a's first
- Then all b's
- Same number of a's and b's

Valid strings:

ab
aabb
aaabbb

Invalid:

aabbb
aaabb
abab

Step 1: Assume A is regular

We **pretend** it's regular.

So:

There exists a pumping length P

Step 2: Choose a string S

We must choose S such that:

$$|S| \geq P$$

Choose:

$$S = a^P b^P$$

Example (if $P = 4$):

$$S = aaaabbbb$$

Length:

$$|S| = 8 \geq 4$$

✓ Good.

Step 3: Split $S = xyz$

Here is the **key idea**:

Because:

$$|xy| \leq P$$

And the **first P symbols are all a's**

y contains only a's

There are NO b's in y.

This is forced. No choice.

General split form:

$$\begin{aligned} x &= a^k \\ y &= a^m \quad (m \geq 1) \\ z &= a^{(P-k-m)} b^P \end{aligned}$$

Step 4: Pump y

Now we test condition 1.

Try $i = 2$

$$xy^2z = a^{(P+m)} b^P$$

Example:

Original: aaaabbbb

Pumped: aaaaabbbb

Counts:

- $a's = P + m$
- $b's = P$

✗ Numbers are NOT equal

✗ String is NOT in $\{a^n b^n\}$

Contradiction

The pumping lemma said:

xy^iz must be in A for all $i \geq 0$

But we found:

$xy^2z \notin A$

So the assumption is false.

Final Conclusion

$A = \{a^n b^n\}$ is NOT regular

Because:

- Pumping breaks the balance
- DFA cannot count and compare
- Finite memory is not enough

Regular Expressions

What a Regular Expression really is

A **regular expression (RE)** is just a **formula to describe patterns of strings**.

Think:

- DFA / NFA $\rightarrow$ machines
- Regular Expression $\rightarrow$ **pattern description**

Same power, different view.

Important:

- **Theory RE $\neq$ programming regex**
- In theory, we only have:
 - **Union**
 - **Concatenation**
 - **Kleene star**
- No lookahead, no backreferences, no fancy stuff

Building blocks of Regular Expressions

Basic ones

If Σ is the alphabet:

- $a \in \Sigma \rightarrow$ matches the string "a"
- $\varepsilon \rightarrow$ matches **empty string**
- $\emptyset \rightarrow$ matches **nothing**

How we build bigger ones

If R1 and R2 are regular expressions:

Operation	Meaning
$R1 \cup R2$	either R1 or R2
$R1 . R2$	R1 followed by R2
$R1^*$	repeat R1 0 or more times

That's it. No magic.

Now let's decode expressions

1. 0^*10^*

Break it:

- $0^* \rightarrow$ any number of 0s
- $1 \rightarrow$ exactly one 1
- $0^* \rightarrow$ any number of 0s

Strings:

1
01
100
0001000

✗ Not allowed:

(no 1)
more than one 1

Exactly one 1, surrounded by 0s

2. $\Sigma^*(001)\Sigma^*$

Meaning:

- $\Sigma^* \rightarrow$ anything
- $(001) \rightarrow$ must appear
- $\Sigma^* \rightarrow$ anything after

Any string that **contains 001 as a substring**

Examples:

001
1001
0000100111

3. $(\Sigma\Sigma\Sigma)^*$

- $\Sigma\Sigma\Sigma \rightarrow$ any **3 symbols**
- $^* \rightarrow$ repeat blocks of 3

So length must be:

0, 3, 6, 9, ...

Examples:

ε

abc

010101

Length 1, 2, 4, 5 $\rightarrow$ NOT allowed

4. $1^* \emptyset$

Key fact:

- $\emptyset$ means **no strings**

So:

anything $\circ$ nothing = nothing

Result:

$\emptyset$

5. $\emptyset^*$

This one feels tricky but is simple.

- $*$ allows **zero repetitions**
- Zero repetitions of $\emptyset = \varepsilon$

Result:

$\{\varepsilon\}$

6. $(0 \cup \varepsilon)1^*$

Break it:

- $(0 \cup \varepsilon) \rightarrow$ either 0 or nothing
- $1^* \rightarrow$ any number of 1s

Examples:

ε

1

11

0

01

0111

✗ Not allowed:

00

10

At most **one 0**, only at the beginning

Takeaway

- **Every regular language**
→ can be described by a **regular expression**
- **Regular expressions = DFA/NFA power**
- They are just **different representation**

Remember: DFA recognizes, RE describes.

The Connection – Generalized NFA (GNFA)

You already know this:

- **Regular language $\Leftrightarrow$ DFA $\Leftrightarrow$ Regular Expression**

So:

- If something is regular $\rightarrow$ a DFA can accept it
- And if a DFA accepts it $\rightarrow$ we can write a **regular expression** for it

The question is:

How do we convert a DFA into a Regular Expression?

Answer: **We don't do it directly.**

We use a helper machine called **GNFA**.

The 2-step idea

DFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

GNFA is just a **bridge**. Nothing more.

What is a GNFA?

A **GNFA (Generalized NFA)** is:

- Like an NFA
- BUT instead of symbols (0, 1, a, b) on transitions
- **Each transition is a REGULAR EXPRESSION**

Example idea (not syntax yet):

q1 -- (0|1)* --> q2

That label (0|1)* is a **regex**, not a single symbol.

Why do we need GNFA?

(If the note in this section feels a bit too complex or not well explained, please read below section “**GNFA for IS Students**”)

Because:

- DFA transitions = single symbols
- Regular expressions = patterns

GNFA lets us:

- Slowly **merge states**
- Combine paths

- And finally get **one regex** from start to accept

Important:

GNFA is **NOT** for real machines

GNFA is **ONLY** a math / proof tool

Sir will **never** ask you to “run” a GNFA. (At least I believe so)

GNFA structure

GNFA is written as:

$$(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$$

You’ve seen this before — it looks like DFA/NFA, but with tweaks.

Let’s decode each part:

Q — States

Same idea: a finite set of states.

BUT:

- GNFA has **exactly ONE start state**
- GNFA has **exactly ONE accept state**

(We force this even if DFA had many)

Σ — Alphabet

Same alphabet as the DFA

Nothing new here.

δ — Transition function (THIS is the twist)

In DFA:

$$\delta : Q \times \Sigma \rightarrow Q$$

In GNFA:

$$\delta : (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow R$$

What this **means in human language**:

- Transitions:
 - Do **not** go INTO the start state
 - Do **not** go OUT OF the accept state
- Each transition is labeled with a **regular expression (R)**

So instead of:

```
q1 -- a --> q2
```

You have:

```
q1 -- (a|b)* --> q2
```

qstart — Start state

- One single start state
- No incoming transitions to it

qaccept — Accept state

- One single accept state
- No outgoing transitions from it

Intuition

Think of GNFA like this:

“Every path from start to accept describes some strings.

Let’s compress all paths into **one giant regular expression.**”

GNFA lets us:

- Remove states one by one
- Update regex labels
- Until only **start** → **accept** remains

That label = **the final regular expression**

Summary

- Regular languages can be described by **DFA or regex**
- To convert DFA → regex:
 1. Convert DFA → GNFA
 2. Eliminate states → get regex
- GNFA:
 - Uses **regular expressions on transitions**
 - Has **one start** and **one accept**
 - Is a **theoretical tool only**

GNFA For IS Students

Step 0: Forget the fancy words

Ignore **GNFA**, **regex**, **compression**, etc.

Just answer this:

☞ **What does a DFA do?**

A DFA:

- starts at **start state**
- moves through states
- ends at **accept state**
- each **path** spells a **string**

Every **path** = **one possible string**

All paths together = **the language**

Step 1: “Every path from start to accept describes some strings”

This literally means:

```
start --a--> q1 --b--> accept
```

This path accepts:

"ab"

Another path:

```
start --a--> q1 --c--> q2 --b--> accept
```

This path accepts:

"acb"

So the language is:

{ ab, acb, ... }

Nothing fancy yet.

Step 2: What does GNFA change?

In a **GNFA**:

- arrows are **not single symbols**
- arrows are **regular expressions**

So instead of:

```
q1 --a--> q2
```

We allow:

$q1 \xrightarrow{a|b} q2$

That arrow already represents **many paths at once**.

GNFA = “bundle many paths into one arrow”

Step 3: “Compress all paths into one giant regular expression”

Think like this:

If the DFA has:

- many states
- many paths
- many strings

We want:

ONE regex that describes ALL those paths

So instead of listing:

ab
acb
aab
...

We want something like:

$a(b|cb|ab)^*$

Same meaning. One description.

That’s what “compress” means.

Step 4: “Remove states one by one”

This is the key GNFA trick.

Suppose we have:

$q1 \xrightarrow{R1} q2 \xrightarrow{R2} q3$

If we **remove q2**, we replace it with:

$q1 \xrightarrow{R3} q3$

Because:

- to go from q1 to q3
- you must go through q2

- so the regex is **concatenation**

If there was also a direct path:

q1 --R3--> q3

Then we combine:

q1 --(R3 U R1R2)--> q3

So:

- removing a state
- updates regex labels
- but keeps the **same language**

Step 5: “Until only start → accept remains”

We keep removing **intermediate states**.

Eventually, only this remains:

start --R--> accept

That **R** is:

- the regular expression
- that describes **every possible path**
- from start to accept

Final meaning

GNFA is just a **mechanical way** to:

- take a DFA
- merge paths step by step
- and end with **one regex**

No magic. No guessing. Just algebra on arrows.

Example DFA Validation

- **States:** Q1, Q2
- **Start:** Q1
- **Accept:** Q2
- **Alphabet:** $\Sigma = \{0, 1\}$
- **Transitions:**

- $Q1 \rightarrow 0 \rightarrow Q2$
- $Q1 \rightarrow 1 \rightarrow Q1$
- $Q2 \rightarrow 1 \rightarrow Q2$
- $Q2 \rightarrow 0 \rightarrow Q1$

Convert DFA $\rightarrow$ GNFA

GNFA rules:

- Add **new start state (qs)** and **new accept state (qa)**
- Label each transition with **regex** for the original DFA moves
- ϵ -transitions: $qs \rightarrow Q1 = \epsilon$, $Q2 \rightarrow qa = \epsilon$

GNFA transitions as regex:

From $\rightarrow$ To	Regex
$Q1 \rightarrow Q1$	1
$Q1 \rightarrow Q2$	0
$Q2 \rightarrow Q1$	0
$Q2 \rightarrow Q2$	1
$qs \rightarrow Q1$	ϵ
$Q2 \rightarrow qa$	ϵ

State elimination method

Idea: remove intermediate states one by one and update the regex.

- Let's **eliminate Q1 first**:
 - New transition from $Q2 \rightarrow Q2$:
 - Original: 1
 - Plus: path $Q2 \rightarrow Q1 \rightarrow Q2$: $0 (1^* 0)$
 - So new $Q2 \rightarrow Q2 = 1 + 0 1^* 0$
- Now $Q2 \rightarrow qa$ stays ϵ
- Finally, start state $qs \rightarrow Q2 = \epsilon 0 1^*$? Let's simplify carefully.

Final Regular Expression

After eliminating Q1:

- **Start qs → Accept qa** = $0(1^*0 + 1)^*$
- Explanation:
 1. $0 \rightarrow$ move from Q1 to Q2 initially
 2. $(1^*0 + 1)^* \rightarrow$ loop over Q2 (stay on 1, or toggle via $0-1^*0$ pattern)

This regex generates **all strings with odd number of 0's**, matching the DFA.

Change alphabet to $\Sigma = \{0, 1, 2\}$

Now your alphabet has a **new symbol: 2**.

Problem: The DFA has **no transition defined for 2**.

- DFA rules: every state must have **exactly one transition for each symbol**.
- Since there's no transition for 2, the DFA is **incomplete** for this alphabet.

What would happen?

1. You either:
 - Add transitions for 2 (make a new DFA for $\Sigma = \{0, 1, 2\}$)
 - Or treat missing transitions as “dead” (go to a non-accepting trap state)
2. **Answer changes:** yes, because now the DFA doesn't define behavior for all strings over $\Sigma = \{0, 1, 2\}$.

Intuition

- DFA is **strict about the alphabet**.
- If the alphabet grows, you must **update transitions**, otherwise the DFA is **invalid**.

Context-Free Languages (CFLs)

Why Regular Grammars Are Not Enough

- **Regular grammars** (like DFAs/NFAs) can generate repetition (e.g., aaa, 1111), but they **cannot handle nested or matched patterns**.
- Things they **cannot do**:
 1. **Nested structures**: $()$, $(())$, $\{\{\}\}$
 2. **Matching symbols**: $0^n 1^n$ (equal number of 0s followed by 1s)
 3. **Programming syntax**: nested if, loops, function calls
- **Why?** Regular languages have **no memory**, so they can't remember how many things happened before.

Conclusion: To describe languages that need **matching or nesting**, we need **Context-Free Languages (CFLs)**.

Definition of Context-Free Grammar (CFG)

A **CFG** is a 4-tuple: (V, Σ, R, S)

Where:

1. **V – Variables / Non-terminals**
 - Abstract symbols representing patterns.
 - Examples: $\langle \text{expr} \rangle$ (expression), $\langle \text{stmt} \rangle$ (statement), $\langle \text{term} \rangle$ (part of an expression)
2. **Σ – Terminals**
 - Actual symbols that appear in strings of the language.
 - Examples in programming: +, *, =, a, b, 0, 1
3. **R – Rules / Productions**
 - Each rule looks like: $A \rightarrow \alpha$
 - LHS A is **one non-terminal**
 - RHS α can be a combination of terminals and non-terminals
4. **S – Start Symbol**
 - Non-terminal where derivation begins

Example: $a^n b^n$ (same number of a's followed by b's)

$S \rightarrow aSb \mid \epsilon$

- Interpretation:

- Either: add a then S then b
- Or: stop (ϵ)

Why CFLs are Needed for Programming Languages

- Programming languages often have **nested structures**:
 - Expressions inside parentheses
 - If-else blocks inside other if-else blocks
 - Loops inside loops
- **Regular grammars can't track this**, but CFLs can, because CFGs can **use recursion**:
 - Non-terminal can reference itself, allowing **unbounded nesting**

Example: Nested parentheses

$S \rightarrow (S) \mid \epsilon$

- Generates: $()$, $(())$, $((()))$, etc.

Techniques to Design CFGs

1. Union of simpler CFLs

- If a language is made of simpler sub-languages, build CFGs for each and combine.
- Example: $L = \{0^n 1^n\} \cup \{1^n 0^n\}$
 - CFG1: $S_1 \rightarrow 0S_1 1 \mid \epsilon$
 - CFG2: $S_2 \rightarrow 1S_2 0 \mid \epsilon$
 - Combine: $S \rightarrow S_1 \mid S_2$

2. CFG from Regular Languages

- Every DFA can be converted to a CFG:
 - Each DFA state $\rightarrow$ non-terminal
 - DFA transitions $\rightarrow$ production rules
 - Example: DFA transition $\delta(q_i, a) \rightarrow q_j$ becomes CFG rule: $R_i \rightarrow aR_j$

3. Handling unbounded memory / recursion

- Some CFLs need to **remember information** to match things (like $a^n b^n$)
- CFG allows **recursive rules**, enabling **stack-like memory**
- Example: matching parentheses, nested expressions, repeated patterns

4. Combining techniques

- Often, you'll use a mix of union, recursion, and regular-to-CFG conversion to create the full CFG.

Key Points

- **CFGs allow recursion**, so they can handle nested structures, unlike regular grammars.
- **Recursive production rules** are the main tool for matching and nesting.
- CFLs can represent **programming language syntax**, unlike regular languages.
- **Terminals** are the actual symbols, **non-terminals** are placeholders or patterns.
- **Start symbol** marks where derivation begins.

Examples

1. Matching $a^n b^n$

$$S \rightarrow aSb \mid \epsilon$$

- Generate: ϵ , ab, aabb, aaabbb, etc.

2. Nested parentheses

$$S \rightarrow (S) \mid \epsilon$$

- Generate: $()$, $((()))$, $((()))()$, etc.

3. Combination

Language with either $0^n 1^n$ or $1^n 0^n$:

$$\begin{aligned} S &\rightarrow S1 \mid S2 \\ S1 &\rightarrow 0S1 \ 1 \mid \epsilon \\ S2 &\rightarrow 1S2 \ 0 \mid \epsilon \end{aligned}$$

Ambiguity in Context-Free Grammars (CFGs)

What is Ambiguity?

- A CFG is **ambiguous** if there exists **at least one string** that can be generated in **two or more different ways**.
- “Different ways” means **different parse trees** or **different sequences of derivation steps**.

Intuition:

- Think of a sentence in English: "I saw the man with a telescope"
 - Parse 1: *I used a telescope to see the man*
 - Parse 2: *The man I saw had a telescope*
- Same string, **two meanings**, so the grammar for this sentence is **ambiguous**.

Why Ambiguity is a Problem in Programming Languages

- Ambiguous grammar → **multiple parse trees** → **multiple interpretations**
- **Compilers get confused**: they need **one clear structure** to generate correct code.
- Can create **logical errors** in evaluating expressions if the parser chooses the wrong tree.

Example: Arithmetic expressions

```
expr → expr + expr | expr * expr | id
```

- String: $A + B * C$
- Two parse trees:
 1. $(A + B) * C$ → add first, then multiply
 2. $A + (B * C)$ → multiply first, then add
- Without rules for **operator precedence**, the grammar is **ambiguous**.

How to Fix Ambiguity

1. **Rewrite the grammar unambiguously**
 - Introduce **different non-terminals** for different precedence levels.
 - Example: separate expr for addition and term for multiplication.

```
expr → expr + term | term
term → term * factor | factor
factor → id
```

- Now $A + B * C$ always parses as $A + (B * C)$
- The **parse tree is unique**, grammar is **unambiguous**

2. Some languages are **inherently ambiguous**

- No matter how you write the CFG, some strings always have multiple valid trees.
- Example: certain natural languages or programming constructs

Key Points

- Ambiguity = **multiple parse trees for the same string**
- Problem for **compilers** → must have unambiguous interpretation
- Fixed by **rewriting grammar** with **operator precedence**, **associativity**, or **separate non-terminals**

Chomsky Normal Form (CNF)

What is Chomsky Normal Form (CNF)?

Chomsky Normal Form (CNF) is a **restricted format** for writing **Context-Free Grammars (CFGs)**.

CNF does **NOT** change what the language means.

It only changes **how the rules are written**.

Why do we need CNF?

- Parsing algorithms (like **CYK**) require CNF
- Proofs about CFGs become easier
- Grammar structure becomes uniform and predictable

CNF is for theory and algorithms, not for humans writing code

What rules are allowed in CNF?

A CFG is in **CNF** if **every production rule** is **only** one of these forms:

Allowed forms

1. **Two non-terminals**

$$A \rightarrow B C$$

2. **One terminal**

$$A \rightarrow a$$

3. **Empty string (special case)**

$$S_0 \rightarrow \varepsilon$$

Only the **start symbol** may produce ε .

NOT allowed in CNF

Rule	Why not allowed
$A \rightarrow B$	Unit rule
$A \rightarrow aB$	Terminal + non-terminal
$A \rightarrow ABC$	Too many symbols
$A \rightarrow \varepsilon$ (if A is not start)	ε -rule

Why CNF looks “unnatural”

Real grammars look like:

$$S \rightarrow aSb \mid \varepsilon$$

CNF wants:

```
S → A B
A → a
B → S b
```

- Same language
- More structure
- Less freedom

That's the tradeoff.

CNF Conversion — WHY each step exists

Now the **important part**.

Every step exists because **something will break otherwise**.

STEP 1: Add a new start variable

What we do

If original start symbol is S, create:

```
S0 → S
```

Now:

- Start symbol = S₀
- S becomes a normal variable

WHY this step exists

Reason 1: ε-removal can destroy the grammar

Example:

```
S → ε
```

If we remove ε:

- Grammar generates **nothing**
- Language destroyed

With new start symbol:

```
S0 → S
S → ε
```

Now:

- Even if $S \rightarrow \epsilon$ is removed carefully
- S_0 still exists
- Language is preserved

S_0 is a safety wrapper

Reason 2: CNF forbids start symbol on RHS

CNF rule:

Start symbol must NOT appear on the right-hand side

Example (illegal in CNF):

$A \rightarrow S$

Fix:

- Make S_0 the start symbol
- S can now appear anywhere legally

Intuition

“Before surgery, put the grammar in a protective case.”

That case is $S_0 \rightarrow S$.

STEP 2: Eliminate ϵ -rules (except start)

What is an ϵ -rule?

$A \rightarrow \epsilon$

CNF dislikes ϵ because:

- It creates **invisible structure**
- Parsing becomes ambiguous and messy

CRITICAL WARNING

You CANNOT just delete ϵ -rules.

That **changes the language**.

Understand ϵ -removal without breaking the language

We start with this grammar:

$S \rightarrow a S b$

$S \rightarrow \epsilon$

First: What language does this grammar generate?

Let's **generate strings**, not theory.

Using $S \rightarrow \epsilon$

- $S \Rightarrow \epsilon$
So ϵ is in the language ✓

Using $S \rightarrow a S b$

Replace the inner S:

- $S \Rightarrow a S b$
- $S \Rightarrow a (\epsilon) b$
- Result: **ab**

Do it again:

- $S \Rightarrow a S b$
- $\Rightarrow a (a S b) b$
- $\Rightarrow a a \epsilon b b$
- Result: **aabb**

Again:

- Result: **aaabbb**

Final language is:

$\{ \epsilon, ab, aabb, aaabbb, \dots \}$

This is **$a^n b^n$** , including ϵ .

Why do we want to remove ϵ -rules?

CNF (and parsing algorithms) **do not like** rules like:

$S \rightarrow \epsilon$

Because:

- ϵ is invisible
- Parser doesn't know *when* something disappears
- Algorithms break or get complicated

So CNF says:

“Get rid of ϵ -rules (except maybe start symbol)”

What goes WRONG if we delete ϵ blindly?

Suppose you **just delete**:

$S \rightarrow \epsilon$

Now grammar is:

$S \rightarrow a S b$

Let's try to generate a string.

- $S \Rightarrow a S b$
- But S has **no way to stop**
- Infinite recursion
- No string ever finishes

Result:

Language becomes **EMPTY**

So yes — **ϵ -removal CAN destroy the language**

That's why your confusion is totally valid.

So what is the CORRECT idea?

Key fact:

S could disappear before.

If we remove that ability, we must **manually allow the effect of disappearing**.

What does “ S disappears” actually mean?

Look at this rule:

$S \rightarrow a S b$

If S disappears (because of $S \rightarrow \epsilon$), then:

$a S b$
 $\downarrow$
 $a \epsilon b$
 $\downarrow$
 ab

ab exists only because S could vanish

So when we remove ϵ , we must **explicitly add the result of that vanishing**.

Step-by-step ϵ -removal

Original rules:

```
S → a S b
S → ε
```

Step A: Identify nullable variables

A variable is **nullable** if it can become ϵ .

- $S \rightarrow \epsilon$
S is nullable

Step B: Find where nullable variables appear

Where does S appear?

```
S → a S b
      ↑
```

Step C: Create new rules by removing nullable occurrences

Original:

```
S → a S b
```

Remove S (pretend it vanishes):

```
S → a b
```

This rule **represents what used to happen via ϵ**

Step D: Now safely delete ϵ -rule

Delete:

```
S → ε
```

Final grammar (ϵ removed correctly)

```
S → a S b | ab
```

Verify language is PRESERVED

Let's generate strings again.

Base case:

$S \rightarrow ab$

✓ ab

Recursive:

```
S → a S b
  → a (ab) b
  → aabb
```

Again:

```
→ aaabbb
```

ϵ ?

- ϵ is no longer generated ✗
- That's OK in CNF **unless S is start symbol**
- If ϵ is required, we handle it using a **new start symbol** (Step 1 of CNF)

The CORE IDEA

Removing ϵ means replacing “disappearing variables” with the strings they used to produce when they disappeared.

Not deleting.

Replacing the effect.

(Back to CFG to CNF Conversion Steps)

STEP 3: Eliminate unit rules

What is a unit rule?

```
A → B
```

Why remove them?

- They add **no terminals**
- They add **no structure**
- They are just useless jumps

Example

Original:

```
A → B
B → aC
```

Replace with:

$$A \rightarrow aC$$

- Language stays the same
- Grammar becomes simpler
- CNF requirement satisfied

Intuition

“If A only points to B, skip B.”

STEP 4: Convert remaining rules into CNF shape

Now we **force** every rule into:

$$A \rightarrow BC$$
$$A \rightarrow a$$

Case 1: RHS too long

Bad rule:

$$A \rightarrow B C D$$

Fix:

$$A \rightarrow B X$$
$$X \rightarrow C D$$

Why?

- CNF allows **only two** non-terminals

Case 2: Terminal mixed with non-terminal

Bad rule:

$$A \rightarrow aB$$

Fix:

$$A \rightarrow X B$$
$$X \rightarrow a$$

Why?

- CNF forbids mixing terminals with variables

Final Big Picture

- **CNF does not change the language**
- **CNF forces strict rule shapes**
- Every step exists to:
 - protect the language
 - simplify parsing
 - enable algorithms

Summary

Chomsky Normal Form is a restricted CFG format where every production is either $A \rightarrow BC$, $A \rightarrow a$, or $S_0 \rightarrow \epsilon$, obtained through systematic elimination of ϵ -rules, unit rules, and long or mixed productions without changing the language.

Pushdown Automata

Why PDA exists

- Regular languages (DFA/NFA) **cannot handle nesting or matching**.
Example: $0^n 1^n \rightarrow$ same number of 0s followed by 1s. DFA can't remember how many 0s it saw.
- **Context-Free Grammars (CFG)** can describe these patterns.
- But if you want a **machine** to recognize CFGs, you need **memory**.
That's where **PDA** comes in.
- PDA = NFA + **stack memory**
 - Stack lets the machine “remember unbounded info” (like how many 0s we've seen).

Formal definition of PDA

A PDA is a 6-tuple:

$(Q, \Sigma, \Gamma, \delta, q_0, F)$

Where:

Symbol	Meaning
Q	Set of states
Σ	Input alphabet
Γ	Stack alphabet (symbols you can push/pop)
δ	Transition function: $Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$
q_0	Start state
F	Accept states

What δ really does

- Reads **current state**, **current input symbol** (or ϵ), **top of stack** (or ϵ).
- Chooses **next state** and **stack action** (push/pop/replace).
- Can be **non-deterministic** $\rightarrow$ multiple possible moves.

Intuition:

The PDA reads input like a DFA, but also has a stack to **remember past info** it might need later.

Example: Language $L = \{0^n 1^n \mid n \geq 1\}$

- This is the classic example where DFA fails. Why?
 - DFA cannot remember how many 0s it has seen.
 - PDA can use stack to “count” 0s.

Idea for PDA

1. **Push 0s onto stack** while reading 0s.
2. **For each 1**, pop a 0 from stack.
3. **If stack is empty** exactly when input ends $\rightarrow$ accept.

Step by step

Input = 0011

Step	Input	Stack	Action
1	0011	ε	Read 0 $\rightarrow$ push 0 $\rightarrow$ stack = 0
2	011	0	Read 0 $\rightarrow$ push 0 $\rightarrow$ stack = 00
3	11	00	Read 1 $\rightarrow$ pop 0 $\rightarrow$ stack = 0
4	1	0	Read 1 $\rightarrow$ pop 0 $\rightarrow$ stack = ε
End	ε	ε	Accept!

Works perfectly because the stack remembers the number of 0s.

Question: What if $n \geq 0$?

- Then **empty string ε** is allowed.
- PDA must also **accept ε** (just start $\rightarrow$ accept immediately if input empty).

Example: $L = \{ww^R \mid w \in \{0,1\}^*\}$

- Strings that are **palindromes**.
- PDA strategy:
 1. Push first half of string onto stack.
 2. Read second half, **pop and match** with stack.
 3. If everything matches $\rightarrow$ accept.

Intuition: **stack lets you “mirror” the first half**.

Summary

- PDA = NFA + stack memory
- Stack is what allows **matching, counting, and nesting**, which DFA cannot do.
- PDA is exactly as powerful as CFG: every CFG can be recognized by some PDA, and vice versa.

Equivalence of Context-Free Languages and Pushdown Automata

Statement of Equivalence

A language is **context-free (CFL)** iff (if and only if) it can be recognized by a **pushdown automaton (PDA)**:

1. CFG → PDA

- If a language is described by a **context-free grammar**, then there exists a PDA that can recognize it.
- Intuition: PDA can **simulate the derivations of the CFG**, using the stack to keep track of variables that need expansion.

2. PDA → CFG

- If a language is recognized by a PDA, then there exists a CFG that generates it.
- Intuition: The CFG can **encode the stack behavior** of the PDA into production rules.

What This Means

- Every **context-free language** can be processed by a PDA.
Example: 0^n1^n , palindromes ww^R , balanced parentheses.
- Every language that a PDA can recognize **must be context-free**.
 - No PDA can recognize languages **beyond CFLs** (e.g., $0^n1^n2^n$).

Intuition

Think of it this way:

- **CFG → PDA:**
 - PDA “reads” the input while “guessing” which production rule to apply next.
 - Stack keeps track of **unprocessed non-terminals**.
 - When the stack and input are both empty → accept.
- **PDA → CFG:**
 - For every possible stack move of PDA, create **variables in CFG** representing the “state + stack segment.”
 - Rules simulate the PDA transitions.

Pumping Lemma for CFLs

Purpose

- Some languages **look like they might be CFL**, but they are not.
- **Pumping Lemma for CFLs** helps **prove a language is NOT context-free**.

Think of it this way:

- If a language is CFL, **any sufficiently long string can be “pumped”** in certain positions without leaving the language.
- If no pumping is possible, the language is **not CFL**.

Statement of the Lemma

If A is a **CFL**, there exists a number p (pumping length) such that:

- Any string $s \in A$ with $|s| \geq p$ can be split as:

$$s = uvxyz$$

Where:

1. $uv^i xy^i z \in A$ for **all** $i \geq 0 \rightarrow$ you can repeat v and y any number of times, including 0.
2. $|vy| > 0 \rightarrow$ at least one symbol in v or y (so pumping actually changes the string).
3. $|vxy| \leq p \rightarrow$ only a **small portion of length $\leq p$** is affected.

Intuition

- Imagine a **parse tree of a CFG** for a very long string.
- If the string is long, **some non-terminal appears twice along a path** (pigeonhole principle).
- This repeated non-terminal corresponds to **v and y** , which can be repeated.
- x is the part between the repeats.

Why two pumpable parts (v and y)?

- CFLs can have **nested structures**, so pumping in **two places** preserves the hierarchical relationship.

How to Use It to Prove a Language is NOT CFL

Steps:

1. **Assume** the language A is CFL.
2. Let p = pumping length.
3. Pick a string $s \in A$ with $|s| \geq p$.
4. Consider **all possible splits** $s=uvxyz$ satisfying $|vy| > 0$ and $|vxy| \leq p$.

5. Pump v and y ($i = 0, 2, \dots$).
6. If for **every split**, $uv^ixy^iz \notin A$ for some i , **contradiction!**
7. Conclude: A is **not CFL**.

Example 1: Language

$$A = \{0^n 1^n 2^n \mid n \geq 1\}$$

- Pick p = pumping length.
- Choose $s = 0^p 1^p 2^p$

Step 1: Consider split $s = uvxyz$

- Lemma: $|vxy| \leq p \rightarrow vxy$ **cannot span all three blocks of 0s, 1s, 2s**.
- $|vy| > 0 \rightarrow v$ or y has at least one symbol.

Step 2: Pump $i = 2$

- Example split:

$$u = 0^{p-2}, \quad v = 00, \quad x = 1, \quad y = 1, \quad z = 1^{p-2} 2^p$$

- Pump: uv^2xy^2z

$$uv^2xy^2z = 0^{p-2} 0000 1111^{p-2} 2^p = 0^{p+2} 1^p 2^p$$

- Now $0s \neq 1s \neq 2s \rightarrow$ **not in the language**

Contradiction $\rightarrow$ **language is not CFL**

Visual Intuition

$$s = 000111222 \quad (p=3)$$

Split: $u=0, v=00, x=1, y=1, z=222$

Pump $i=2 \rightarrow uv^2 x y^2 z$

$0 \ 00 \ 00 \ 1 \ 11 \ 222 \rightarrow 0000011222 \quad \times \text{ invalid}$

- Pumping broke the equal counts $\rightarrow$ shows **not CFL**.

Example 2: Language

$$B = \{ww \mid w \in \{0, 1\}^*\}$$

- Strings are **exactly repeated halves**.
- Pick $s = 0^p 1^p 0^p 1^p$

- Any split $s = uvxyz$ with $|vxy| \leq p$ and $|vy| > 0$ affects **only part of the first half**.
- Pump $i=2 \rightarrow$ first half becomes longer, second half stays same $\rightarrow$ halves **no longer match** $\rightarrow$ string not in language

Contradiction $\rightarrow$ **not CFL**

Visual Intuition

```
s = 000111000111 (p=3)

Split: u=00, v=0, x=1, y=1, z=1000111
Pump i=2  $\rightarrow uv^2xy^2z$ 
00 00 1 11 1000111  $\rightarrow$  first half  $\neq$  second half ✖
```

Key Takeaways

- CFL Pumping Lemma = **5 parts**: $uvxyz$
- **Two pumpable parts**: v and y
- **Middle part bounded**: $|vxy| \leq p$
- **Use it to disprove CFL**, not to prove CFL
- Works best on **languages with dependencies across multiple sections**

Turing Machines (TM)

- Invented by **Alan Turing** in **1936**.
- Purpose: a **new computation model** to describe **all kinds of computations**.
- Can model languages that:
 1. Turing Recognizable
 2. Undecidable
 3. Not even Turing recognizable

Main idea: TM is a **finite control + infinite tape** that can read/write symbols.

Components of a TM

A Turing Machine has:

1. **Finite control** – like the states in a DFA/NFA
2. **Tape** – infinitely long memory, divided into cells
3. **Read/write head** – can read symbols, write symbols, and move left or right

Turing Machine Notation

1. **Tape**

- The tape is like an **infinite strip of paper**, divided into **cells**.
- Each cell can hold **one symbol** from a finite set.
- The **input string** is written on the tape.
- All unused cells are filled with a **blank symbol (often ‘_’)**.

2. **Tape Head**

- The TM has a **head** that points to one cell at a time.
- The machine **reads the symbol** under the head.
- It can also **write a new symbol** and **move left or right**.

The tape is the TM's **memory**.

The tape head is like your **pointer** that keeps track of where the machine is working.

Move of a Turing Machine

A **move** is how the Turing Machine acts at one step. It depends on:

- **Current state** (from the finite control)
- **Symbol under the tape head**

In **one move**, the TM does **three things**:

1. Change state

- It can go from the current state to another state (or stay in the same one).

2. Write a symbol

- It can **replace the symbol** under the head with a new symbol (or keep it the same).

3. Move the tape head

- The head can **move one cell left or right**.

Intuition:

- Think of it like reading a letter, deciding what to do next, **writing a reply**, and moving to the next letter.

Formal Definition of a Turing Machine (TM)

A **TM** is described by a **7-tuple**:

$$\text{TM} = \{Q, \Sigma, \Gamma, \delta, q_0, F, R\}$$

Symbol	Meaning
Q	Finite set of states (the “brain” of the TM)
Σ	Input symbols (what you give to the TM; blank symbol is not included)
Γ	Tape symbols (Σ + blank + any extra symbols TM might use)
δ	Transition function (rules that decide what TM does next)
q_0	Start state (where computation begins)
F	Accepting states (TM accepts input if it reaches any of these)
R	Rejecting states (TM rejects input if it reaches any of these)

Transition Function δ

- δ tells the TM: “If you are in state q and see symbol X , then do this...”

Formally:

$$\delta(q, X) = (p, Y, D)$$

Where:

1. $p \rightarrow$ Next state (new “brain state”)
2. $Y \rightarrow$ Symbol to write on the tape (replace X)
3. $D \rightarrow$ Move tape head **Left (L)** or **Right (R)**

Intuition:

- **Current state + current symbol $\rightarrow$ decide next move**
- Example:
 - Current state = q_0 , symbol under head = 1
 - $\delta(q_0, 1) = (q_1, 0, R)$
 - Meaning:
 1. Change state to q_1
 2. Replace the 1 with 0
 3. Move head one cell to the right

Example 1: Language 01^*0

Language: Strings starting with 0, then any number of 1s, and ending with 0

Input Examples: 00, 010, 0110, 011110

Input Not Allowed: 1..., 01...1 (no ending 0)

Step 1: Initialize

- Input: 0 1 1 0
- Tape: 0 1 1 0 $\square \square \square \dots$ (blank symbols to the right)
- Tape head starts at **first cell** $\rightarrow$ scanning 0
- States:
 - q_0 = start
 - q_1 = reading middle 1s
 - q_2 = checking last 0
 - Accept / Reject

Step 2: Move through input

Move 1: Check first 0

- State: q_0
- Symbol under head: 0 $\rightarrow$ correct
- Action: move to q_1 , move head **right**

Tape Visual:

```
[q0]0 1 1 0
  ^ head moves right
```

Move 2-3: Skip all 1s

- State: q1
- Symbols: 1, 1 → keep moving right in q1

Tape Visual:

```
0 [q1] 1 1 0
    ^ head
```

```
0 1 [q1] 1 0
    ^ head
```

Move 4: Last 0 check

- State: q1
- Symbol under head: 0 → move to q2

Tape Visual:

```
0 1 1 [q2] 0
    ^ head
```

Move 5: End of input

- State: q2
- Symbol under head: blank → Accept

Visual:

```
0 1 1 0 [Accept]
```

Intuition: TM scans, verifies pattern, and accepts only correct strings.

Example 2: Language $0^n 1^n$ ($n \geq 1$)

Language: Equal number of 0s followed by 1s

Input Examples: 01, 0011, 000111

Input Not Allowed: 001, 011, 00011

Step 1: Initialize

- Input: 0 0 1 1
- Tape: 0 0 1 1 _ _ _ ...
- Stack-like memory using **marking**: X = used 0, Y = used 1
- Start state: q0

Step 2: Mark first 0

- Head at first 0, state q0
- Replace 0 → X (mark as used)
- Move right → q1 to find first 1

Tape Visual:

[X] 0 1 1

^ head moves right and 0 is now X

Step 3: Move to first 1

- State: q1
- Skip remaining 0s → move right
- Reach first 1 → replace 1 → Y
- Move left → q2 to return to next unmatched 0

Tape Visual:

X 0 [Y] 1

^ head moves left and mark the matching 1 as Y.

Step 4: Return to next 0

- State: q2
- Move left until first unmarked 0
- Found 0 → mark as X → repeat

Tape Visual:

X [X] Y 1

^ head moves right

Step 5: Repeat until all matched

- Process:
 - Mark 0 → X
 - Find 1 → Y
 - Repeat

- After final matching:

XXYY _

- Head moves right past last symbol → blank
- **Accept** because stack-like marking ensures all 0s matched 1s

Lambda Calculus

What Lambda Calculus Actually Is

Lambda Calculus is a **very small mathematical system for computation**.

Instead of using things like:

- variables
- loops
- numbers
- classes

it uses only **functions**.

So in lambda calculus:

- Numbers $\rightarrow$ functions
- True/False $\rightarrow$ functions
- Data structures $\rightarrow$ functions

Everything is built from **functions**.

It was invented by **Alonzo Church** in the 1930s and is one of the theoretical foundations of programming languages.

Languages like:

- **Haskell**
- **Lisp**
- **Scala**

all borrow ideas from it.

The 3 Building Blocks

Every lambda expression is built using **ONLY THREE THINGS**.

Type	Meaning
Variable	a name
Abstraction	function definition
Application	calling a function

These are called **λ -terms**.

Variables

A **variable** is simply a name.

Examples: x, y, z

Nothing fancy.

Think of them like variables in programming.

Example:

x

That alone is a valid lambda expression.

Function Definition (Abstraction)

Functions are written like this:

```
λ variable . expression
```

Example:

```
λx.x
```

Let's break it:

Part	Meaning
λ	"This is a function"
x	input parameter
.	separator
x	body of the function

So:

```
λx.x
```

means

```
function(x) {  
    return x  
}
```

In programming this is called **identity function**.

Example 2

```
λx.x+1
```

Means

```
function(x){  
  return x+1  
}
```

Function Application (Calling a Function)

Applying a function means **giving it a value**.

Syntax:

```
(function argument)
```

Example:

```
(λx.x) y
```

Meaning:

```
function(x){ return x } (y)
```

Result:

```
y
```

Because the function just returns the input.

Associativity

When multiple things appear together:

```
E1 E2 E3
```

It is interpreted as:

```
((E1 E2) E3)
```

So evaluation happens **left to right**.

Example:

```
A B C
```

means

```
((A B) C)
```

Bound vs Free Variables

Bound Variable

A variable **controlled by a λ function**.

Example:

$\lambda x. x$

Here:

x

is **bound** because it belongs to the function.

Free Variable

A variable **not defined inside a function**.

Example:

$(\lambda x. x) y$

Here:

Variable	Type
x	bound
y	free

Why?

Because λx defines x.

But **y is not defined**.

Another Example

$\lambda x. (x y)$

Variable	Type
x	bound
y	free

Reduction (Evaluation)

Lambda calculus **computes by rewriting expressions.**

This rewriting is called **reduction**.

There are two main types.

Alpha Reduction (α -reduction)

This means:

renaming variables

Example:

```
 $\lambda x.x$ 
```

can become

```
 $\lambda y.y$ 
```

Same meaning.

Like changing:

```
function(x){return x}
```

to

```
function(a){return a}
```

Nothing changes.

Purpose: Avoid variable conflicts.

Beta Reduction (β -reduction)

This is the **MOST IMPORTANT** rule.

It means:

apply a function to an argument

Example:

```
 $(\lambda x.x) y$ 
```

Step:

Replace **x** with **y**.

Result:

y

Another Example

$(\lambda x. x+1) 5$

Replace x with 5.

Result:

$5+1$

Example 3

$(\lambda x. x \ x) \ y$

Replace x with y .

Result:

$y \ y$

Church Numerals (Numbers as Functions)

Lambda calculus **does not have numbers**.

So numbers are represented using **function repetition**.

This idea was also created by **Alonzo Church**.

Zero

$0 = \lambda f. \lambda x. x$

Meaning:

Apply function **0 times**.

So result is just x .

One

$1 = \lambda f. \lambda x. f(x)$

Meaning:

Apply **f once**.

Two

$2 = \lambda f. \lambda x. f(f(x))$

Apply **f two times**.

Three

$$3 = \lambda f. \lambda x. f(f(f(x)))$$

Apply f **three times**.

Easy Way to Remember

Number	Meaning
0	apply f zero times
1	apply f once
2	apply f twice
3	apply f three times

Successor Function (Add 1)

We want a function `succ` such that:

$$\text{succ}(2) \rightarrow 3$$

- $2 = \lambda f. \lambda x. f(f(x))$
- $3 = \lambda f. \lambda x. f(f(f(x)))$

Notice: **3 is just 2 plus one more f application.**

Lambda Form

$$\text{succ} = \lambda n. \lambda f. \lambda x. f (n f x)$$

Break it down:

1. $\lambda n \rightarrow n$ is the number we want to increment.
2. $\lambda f \rightarrow f$ is the function we'll apply repeatedly.
3. $\lambda x \rightarrow x$ is the starting value.
4. $f (n f x) \rightarrow$ this is the magic:
 - First compute $n f x \rightarrow$ applies f **n times**
 - Then apply **one more f** $\rightarrow$ gives $n+1$ times

Step-by-Step Example: `succ(2)`

1. Write 2 in lambda form:

$$2 = \lambda f. \lambda x. f(f(x))$$

2. Apply succ to 2:

```
succ 2 = (λn.λf.λx. f (n f x)) (λf.λx. f(f(x)))
```

3. Replace n with 2:

```
λf.λx. f ((λf.λx. f(f(x))) f x)
```

4. Compute the inner part (2 f x):

```
(λf.λx. f(f(x))) f x → f(f(x))
```

5. Apply the outer f:

```
f (f(f(x)))
```

Done! That's **3**, as expected.

Pairs (Two Values Together)

Lambda calculus also builds **data structures using functions**.

A pair (a,b) is defined as:

```
Pair = λa.λb.λf.f a b
```

Meaning:

Store **a and b**, then give them to a function.

Selectors (Get First or Second)

To retrieve values we use selector functions.

First

```
First = λx.λy.x
```

Returns first value.

Second

```
Second = λx.λy.y
```

Returns second value.

Example:

Pair:

```
(a,b)
```

Using:

```
First(a,b) → a
Second(a,b) → b
```

Swap Example

Swap pair $(a,b) \rightarrow (b,a)$.

Lambda expression:

```
λp. Pair (Second p) (First p)
```

Steps:

1. Take pair **p**
2. Extract **second**
3. Extract **first**
4. Create new pair in reverse order

Result:

```
(a,b) → (b,a)
```

Big Picture

Lambda calculus only has:

variables

functions

function calls

Yet from these we can build:

- numbers
- booleans
- pairs
- lists
- recursion
- full computation

That's why it's one of the **foundations of computer science**.